

Hospital Management System (HMS) — Project Report

Project Title: Hospital Management System (HMS) — Full-Stack Clinical Workspace

Group Information:

1. Intisar Ahmed Siyan (22-47483-2)
2. Sayeda Rumaiya Islam (22-47474-2)

1. Project Overview

The Hospital Management System (HMS) is a state-of-the-art, full-stack clinic management web application designed to streamline healthcare scheduling, medical histories, profile updates, and central administrative controls. The system is split into two robust architectural components:

Backend Server (NestJS & TypeORM): A secure, structured REST API built on NestJS using TypeScript, TypeORM, and PostgreSQL. It enforces robust role-based access control (RBAC), manages password encryption via bcryptjs, and implements transaction-safe database workflows to guarantee relational cascading integrity.

Frontend Portal (Next.js & TailwindCSS): A modern, responsive React web application utilizing Next.js App Router.

2. Functional Requirements

Type of Users

Anonymous Guest: Can register a new account, log in, or publicly browse the active clinical specialties.

Patient: Can schedule consultations with available doctors, track upcoming visits, browse their chronological medical records history, and read or print clinical reports.

Doctor: Certified specialists who can view their assigned clinical schedules, filter patient logs, review patients' historical records, and write consultation diagnoses and prescriptions.

Admin: Universal superusers who possess full CRUD authority over medical departments, central oversight of all clinical schedules/records, and account management control over all user profiles.

Common Functionality

Role-Based Authentication: Custom /auth/register and /auth/login pathways with secure JWT validation and session persistence cached in local storage.

Clinic Directory access: Public, anonymous accessibility to read clinical departments (GET /departments) enabling prospective doctors to dynamically choose their specialties during registration.

User Profile Center: A unified editor (src/components/profile/Profile.tsx) where patients, doctors, and admins can edit their registration parameters, generate dynamic name initials avatars, and safely reset passwords.

User Specific Functionality

A. Patient Features

- **Care Portal Dashboard:** Displays upcoming visit counters, dynamic greeting banners, and consultation cards.
- **Scheduling System:** Allows booking future appointments by choosing a specialty department and dynamically filtering certified active clinicians.
- **Timeline Medical History:** Chronological vertical timeline detailing specialist consultation summaries, diagnostic outcomes, and medical guidelines.
- **Printable Medical Reports:** Interactive report views allowing patients to print physical copies of their prescriptions directly via standard browser print commands.

B. Doctor Features

- **Clinical Specialization Badging:** Dynamically locks and renders a doctor's registered clinical specialty badge on their primary dashboard.
- **Appointment Filter Logs:** Interactive toggles sorting medical bookings into Scheduled, Completed, or Cancelled categories.
- **Diagnosis Form Dialog:** An embedded workspace permitting doctors to submit clinical notes and prescribe pharmaceutical regimens, automatically setting appointment statuses to Completed.
- **Deep-Dive History Logs:** Grants doctors complete transparency to read their patient's past medical summaries from other clinic departments.

C. Admin Features

- **Department CRUD Board:** Full dashboard controls to create, update, or permanently delete clinic departments.
- **Records & Schedules Viewer:** A central registry tracking every booking made in the clinic, with an inspector card showing exact diagnoses and prescriptions.
- **User Accounts Registry:** Global panel to audit registered profiles, elevate doctors or patients to Admin privileges, and delete accounts cleanly.

3. Challenges & Difficulties

Dynamic Anonymous Access vs API Shielding:

Challenge: The /departments GET routes were originally locked behind class-level JWT Auth Guards. This blocked anonymous users on the register page from dynamically pulling specialty options.

Solution: Re-architected the DepartmentsController by removing class-level guards and binding them directly on the write methods (POST, PATCH, DELETE). This gracefully opened reading access to anonymous registration queries while securely protecting the database from unsanctioned write operations.

Database Cascading Integrity & Transaction Control:

Challenge: Deleting a doctor or patient account would trigger foreign key constraint violations if they had active or historical clinic records in the database.

Solution: Configured onDelete: 'CASCADE' on both patient and doctor TypeORM relations. Concurrently, the backend deletion service was wrapped in an entityManager.transaction that sweeps associated schedules first, creating a seamless, transaction-safe deletion pipeline.

Next.js Hydration & State Reload Desynchronizations:

Challenge: Hot-reloading client elements alongside asynchronous state calls (such as reading local storage tokens) frequently triggered React hydration mismatches between SSR and CSR.

Solution: Resolved by deferring state evaluation to useEffect hooks and adding dynamic loader states, guaranteeing flawless client-side rendering.